Creamos un programa base que no necesita saber que hace cada plugin, en lugar de que compilemos todo junto desde el principio, el programa recibe la ruta del archivo plugin1.so o plugin2.so, lo abre en la RAM usando(dlopem,dlsym,dlclose), busca la función que necesite y lo ejecuta

```c
#include <stdio.h>

//plugin_run será el nombre que el programa va a buscar en memoria
int plugin_run(const char *argument){
        printf("[Plugin 1 - Eco] Procesando audio/datos: %s\n", argument);
        return 0;
}
```

```c
#include <stdio.h>

//usamos la misma función pero mostramos otro mensaje en el printf
int plugin_run(const char *argument){
        printf("[Plugin 2] Aplicando efectos sobre: %s\n", argument);
        return 1;
}
```

Este es nuestro programa principal runner.c que revisa que le hayamos pasado 2 datos en consola(./runner y la ruta del plugin), luego busca el archivo .so en la carpeta y lo abre en memoria, después rastrea la función que necesita dentro del archiv, llama a la función encontrada y se descarga el puglin de la RAM

```c
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h> //biblioteca para dlopen,dlsym,dlclose


//defino el puntero de la funcion
typedef int (*plugin_run_func)(const char *);

//valido la entrada de argumentos
// si argc < 2(falta plugin), monstramos mensaje de uso y salimos
// si no, guardo la ruta del plugin y sigo la ejecución
int main(int argc, char *argv[]){
        if (argc < 2){
                printf("Uso: %s <ruta_al_plugin.so\n",argv[0]);
                return 1;
        }

        const char *plugin_path = argv[1];

//Cargo el archivo.so en memoria
void *handle = dlopen(plugin_path, RTLD_NOW | RTLD_LOCAL);

//Si el handle es nulo entonces el archivo no existe o no pudo abrirse

if (!handle){
        fprintf(stderr, "Error al abrir el plugin con dlopen: %s\n",dlerror());
        return 1;
}

//busco la funcion dentro del plugin
dlerror();

//busco la direccion exacta de plugin_run
plugin_run_func run = (plugin_run_func) dlsym(handle, "plugin_run");
```

```c
//verifico si dlsym reporto un error
const char *dlsym_error = dlerror();
//Si se produjo
if (dlsym_error){
        fprintf(stderr,"error al buscar el simpolo con dlsym: %s\n",dlsym_error);
        dlclose(handle); //cerramos el archivo abierto
        return 1;
}

//ejecuto la funcion del plugin
int result = run("Audio_Input_Signal");
printf("El plugin devolvio el codigo de estado: %d\n",result);

//libero la memoria
dlclose(handle);
return 0;
}
```

```
juanmapro788@Asustufjuanma:~$ vim plugin1.c
juanmapro788@Asustufjuanma:~$ vim plugin2.c
juanmapro788@Asustufjuanma:~$ gcc -Wall -Wextra -fPIC -shared -o plugin1.so plugin1.c
juanmapro788@Asustufjuanma:~$ gcc -Wall -Wextra -fPIC -shared -o plugin2.so plugin2.c
juanmapro788@Asustufjuanma:~$ vim runner.c
juanmapro788@Asustufjuanma:~$ gcc -Wall -Wextra -o runner runner.c -ldl
juanmapro788@Asustufjuanma:~$ ./runner ./plugin1.so
[Plugin 1 - Eco] Procesando audio/datos: Audio_Input_Signal
El plugin devolvio el codigo de estado: 0
juanmapro788@Asustufjuanma:~$ ./runner ./plugin2.so
[Plugin 2] Aplicando efectos sobre: Audio_Input_Signal
El plugin devolvio el codigo de estado: 1
```

Así vemos que runner no necesitó saber que estuvo haciendo cada puglin, ni estar enlazado a los puglin al compilar.

```
juanmapro788@Asustufjuanma:~$ readelf -Ws plugin1.so | grep plugin_run
     6: 0000000000001119    50 FUNC    GLOBAL DEFAULT   13 plugin_run
    21: 0000000000001119    50 FUNC    GLOBAL DEFAULT   13 plugin_run
juanmapro788@Asustufjuanma:~$ LD_DEBUG=libs ./runner ./plugin1.so
      1447:     find library=libc.so.6 [0]; searching
      1447:      search cache=/etc/ld.so.cache
      1447:       trying file=/usr/lib/x86_64-linux-gnu/libc.so.6
      1447:
      1447:
      1447:     calling init: /lib64/ld-linux-x86-64.so.2
      1447:
      1447:
      1447:     calling init: /usr/lib/x86_64-linux-gnu/libc.so.6
      1447:
      1447:
      1447:     initialize program: ./runner
      1447:
      1447:
      1447:     transferring control: ./runner
      1447:
      1447:
      1447:     calling init: ./plugin1.so
      1447:
[Plugin 1 - Eco] Procesando audio/datos: Audio_Input_Signal
El plugin devolvio el codigo de estado: 0
      1447:
      1447:     calling fini: ./plugin1.so [0]
      1447:
      1447:
      1447:     calling fini:  [0]
      1447:
      1447:
      1447:     calling fini: /usr/lib/x86_64-linux-gnu/libc.so.6 [0]
      1447:
      1447:
      1447:     calling fini: /lib64/ld-linux-x86-64.so.2 [0]
      1447:
```

Ahora ejecutamos esos dos comandos que analizan internamente la carga del puglin1. Con readelf confirmamos que puglin_run figura como un símbolo público y con el otro comando comprobamos que el SO mapea el .so en la RAM únicamente al ejecutar dlopen.

1. Con enlace normal -l, el programa exige la librería antes de arrancar.Con dlopen, el programa arranca solo y abre la librería cuando lo necesita.Por eso no figura en la lista de requeridos.

2. Con -l se busca la función al encender el programa. Con dlopen y dlsym, la función se busca recién en el instante exacto en que tocamos la línea de código que llama a dlsym

3. La biblioteca normal comparte sus funciones con todo el programa. RTLD_LOCAL lo que hace este como si encrrarae al plugin en una caja, sus funciones no se mezclan con las de otros plugins aunque tengan el mismo nombre.

4.

Si no existe, dlopen da error y te dice que no encontró el archivo.

Si falta la función, dlsym da error porque no la encuentra adentro del archivo.

Si la interfaz es incompatible, El programa no se da cuenta, intenta ejecutar la función con los datos mal organizados y se rompe nuestro programa.

5. Si alguien nos manda un .so malicioso, puede borrar archivos o hackearte el programa apenas lo carguemos.